

ESPECIALIZACIÓN EN **INTELIGENCIA DE DATOS** para la Gestión Estratégica

CAPTURA Y ANÁLISIS DE LA INFORMACIÓN DE ENTRADA

Clase 2 · Taller T8001 — Pre-procesado de datos

Docente: Dr. Lautaro Di Bartolo

Asignatura: T8001 — Pre-procesado de datos

Cohorte o comisión: Cohorte 2026

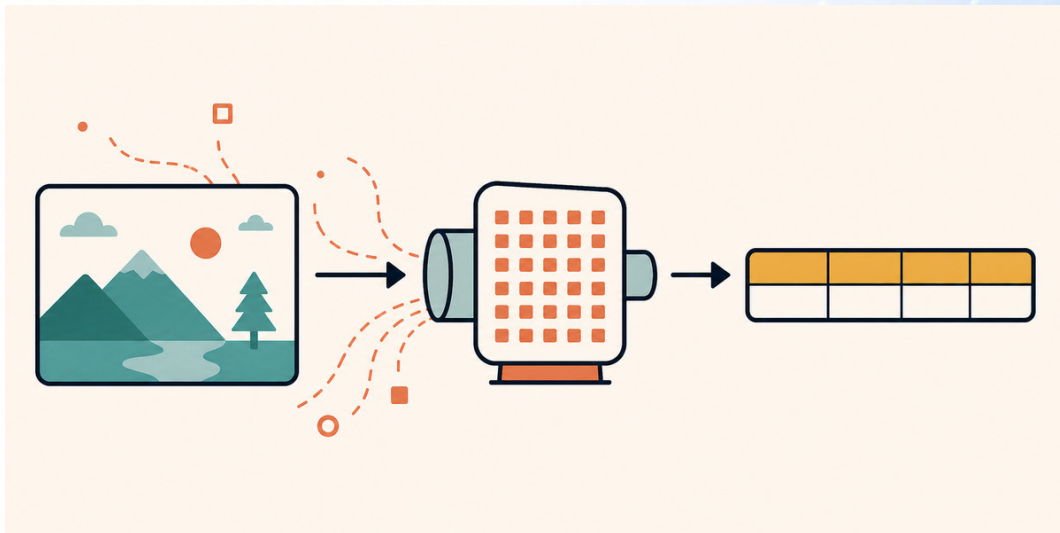


Figura 1. Entre el hecho y la fila hay siempre un instrumento, y ese instrumento deja marcas.

1. INTRODUCCIÓN

1.1 La pregunta que va antes del archivo

La clase 1 abrió archivos y encontró que estaban sucios. Aprendiste a ver que una columna quedó como texto, que un identificador perdió el cero de adelante y que en ingreso falta un valor.

Esta clase hace la pregunta anterior: **¿de dónde salió ese dato y quién decidió cómo se iba a ver?**

Ningún dato aparece solo. Alguien diseñó un formulario, alguien programó un sensor, alguien exportó una planilla. Cada una de esas decisiones deja una marca en el archivo que recibís, y la mayoría de los problemas que vas a arreglar en la clase 3 ya estaban decididos en el momento de la captura.

La clase 1 te enseñó a ver que faltaba un dato. Acá vamos a preguntar **por qué** falta. La respuesta casi nunca está en el archivo, y encontrarla cambia qué se hace con ese faltante.

1.2 Lo que se supone de la clase 1

Este apunte da por sabido lo de la clase anterior. Un CSV es texto plano. `read_csv` devuelve una tabla. Las cinco preguntas de la primera mirada se contestan con `shape`, `columns`, `dtypes`, `isna()` y `duplicated()`. Una tabla usable tiene una variable por columna y un valor por celda. Y el archivo original no se modifica nunca.

No hay herramientas nuevas. Se sigue con pandas y nada más.

Una advertencia sobre las listas. Este apunte tiene muchas listas cerradas, y no hay que memorizarlas todas. Las dos que se usan siempre son los seis pasos del perfilado, de la sección 7.1, y las ocho columnas del diccionario, de la sección 6.5. Las demás son para reconocer un problema cuando aparece, no para recitar.

1.3 El relevamiento de esta clase

Trabajamos sobre el mismo relevamiento de hogares de la clase 1, ahora con doce filas en lugar de cuatro. Sale la columna ambientes y entran tres nuevas: cobertura_salud, satisfaccion y servicios. Es un dato inventado, y eso es a propósito: así entra entero en una página y todo lo que digamos de él se puede verificar mirándolo.

La ficha del conjunto es esta. **Quién lo produjo:** el área de Desarrollo Social de un programa provincial, que releva las tres localidades. **Cuándo:** marzo de 2025. **Cómo:** encuestadores en terreno, con un formulario digital en tablet. **Para qué:** decidir dónde abrir un centro de salud.

id	localidad	personas	fecha_visita	ingreso	cobertura_salud	satisfaccion	servicios
1	Ramallo	3	2025-03-14	185000	si	buena	agua;luz;gas
2	Córdoba	5	2025-03-15	240500	no	regular	agua;luz
3	Concepción	2	2025-03-15		NULL	NULL	luz
4	Ramallo	4	2025-03-16	198000	si	buena	agua;luz;gas
5	ramallo	1	2025-03-16	120000	NULL	NULL	luz
7	Córdoba	6	2025-03-17	310000	no	mala	agua;luz;gas;cloacas
8	Concepción	3	2025-03-17	175000	si	regular	agua;luz
9	Ramallo	2	2025-03-18	999999	no	mala	luz;gas
10	Córdoba	4	2025-03-19	205000	NULL	NULL	agua;luz;gas
11	Concepción	7	2025-03-20	260000	si	buena	agua;luz;gas;cloacas
13	Ramallo	3	2025-03-20	190000	NULL	NULL	agua;luz
14	Córdoba	2	2025-03-21	168000	no	regular	agua

Esa tabla tiene defectos plantados a propósito. No hace falta buscarlos ahora: las secciones que siguen los van encontrando de a uno.

Hay dos que cuesta verlos, y conviene mirarlos ya:

- **La numeración de id llega hasta 14 y hay doce filas.** Faltan el 6 y el 12.
- **En la fila con id 9, la localidad dice Ramallo con un espacio al final.** En la pantalla es indistinguible de las otras. Para la máquina es otra localidad.

2. DE DÓNDE SALE UN DATO

Cuando te piden "un análisis de los datos del municipio", el dato no está en un lugar. Está repartido en sistemas que nacieron en momentos distintos, con dueños distintos y con propósitos distintos. Lo más probable es que ninguno de esos sistemas se haya diseñado para vos.

2.1 Fuentes internas y externas

La diferencia que importa no es dónde está el servidor. Es **cuánto control tenés sobre la definición del dato**.

Qué mirás	Interna	Externa
Cambiar la captura	Negociando	No
Saber qué significa	A veces	Si hay diccionario
Sesgo principal	Del proceso	Del que publica

Una fuente interna es la que produce tu propia organización. Podés sentarte con el área que la carga y pedir que agregue un campo o que valide una respuesta. Cuesta, pero se puede.

Una fuente externa la produce otro. No podés cambiar nada: solo podés entender qué te dieron y decidir si sirve. El [Portal Nacional de Datos Públicos](#) es una fuente externa, y la planilla que te manda un compañero por correo es una fuente interna aunque llegue por el mismo camino. De ahí sale la última fila de la tabla: el sesgo de nuestro relevamiento es a qué hora fue el encuestador, y del portal es qué decidió publicar el organismo. Por lo primero podés preguntar; por lo segundo, no.

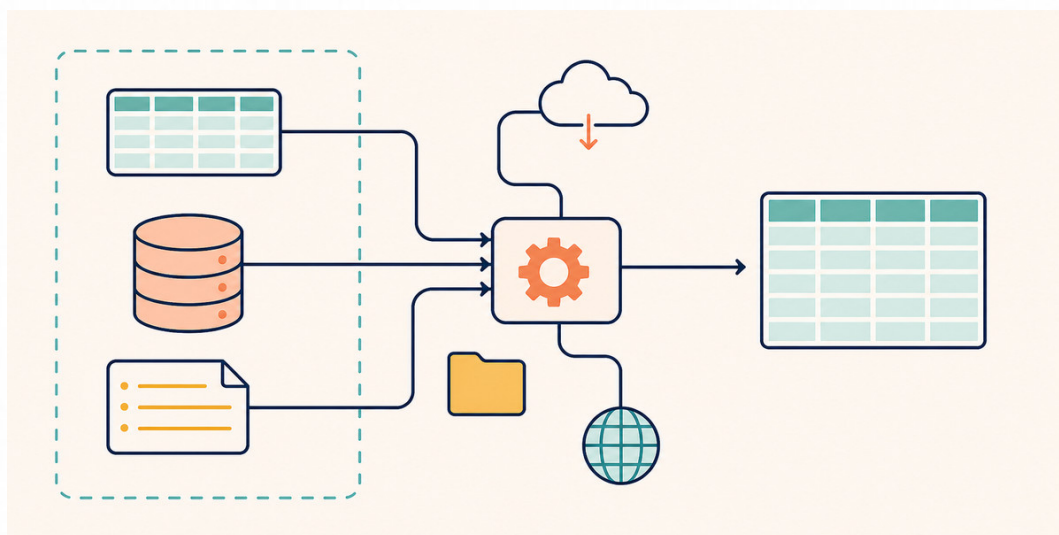


Figura 2. El dato llega desde sistemas que nacieron por separado y con otro propósito.

2.2 Seis tipos de fuente y la trampa de cada uno

De cada tipo, lo único que hay que recordar es qué deja afuera:

- **Sistemas transaccionales** (ventas, turnos, facturación). Solo existe lo que pasó por el sistema. Una venta en efectivo sin ticket no está.
- **Registros administrativos** (legajos, padrones). Se actualizan cuando el ciudadano tiene un motivo para actualizarlos. Un domicilio puede tener diez años.
- **Sensores y telemetría**. El sensor no sabe que se descalibró. Sigue informando, y el número que informa es plausible.
- **Logs de sistemas**. Miden interacción con la pantalla, no intención. Y se rotan: lo viejo se borra solo.
- **Formularios y encuestas**. Contesta quien quiere, y contesta lo que le parece aceptable decir.
- **Web y datos abiertos**. El organismo publica lo que decidió publicar, con el recorte que decidió. [Buenos Aires Data](#) y el portal nacional son de este tipo.

El relevamiento de esta clase es del quinto tipo, y por eso vamos a encontrar los problemas de ese tipo.

Tres de estos seis vuelven a aparecer en la sección 3.3, y ahí se los mira de otra manera: no como lugares donde el dato está guardado, sino como procedimientos que lo produjeron.

2.3 Las tres preguntas que van antes del análisis

1. **¿Qué mide realmente este dato?** No lo que dice el nombre de la columna: qué evento concreto tuvo que ocurrir para que aparezca esta fila. En el relevamiento, la respuesta es "un encuestador llegó a la puerta y alguien le abrió".
2. **¿Quién queda afuera?** Toda fuente tiene una población que no alcanza. Nombrala. Acá queda afuera el hogar donde no había nadie a las tres de la tarde.
3. **¿Cada cuánto y hasta cuándo se actualiza?** Un dato que se dejó de actualizar en 2019 no es un dato actual, aunque lo hayas descargado hoy. Acá la respuesta es "una sola vez, en marzo de 2025, y no se repite": es la **oportunidad** de la clase 1, sección 3.7.

Las tres se contestan antes de abrir el archivo: con la ficha, con el diccionario, o preguntándole a quien lo produjo.

3. CÓMO SE CAPTURÓ

3.1 Qué es un mecanismo de captura

Un **mecanismo de captura** es el procedimiento concreto por el cual un hecho del mundo se convierte en una fila de una tabla.

No confundas esto con la sección 2.2. Allá clasificamos **de dónde viene** el dato, o sea qué sistema lo tiene guardado. Aquí clasificamos **cómo se produjo**, o sea qué procedimiento lo creó. Las dos listas se cruzan y no son la misma lista: un sistema de turnos puede alimentarse de un formulario, y un portal de datos abiertos puede estar publicando lo que salió de una planilla cargada a mano.

La tesis de esta clase, en una línea:

Cada mecanismo de captura produce su propia familia de errores, y esos errores son predecibles. Si sabés cómo se capturó, sabés qué buscar antes de abrir el archivo.

3.2 Error y sesgo no son lo mismo

Dos palabras que conviene separar desde el principio:

- **Error.** Una desviación sin dirección. El error grosero se detecta mirando el dato: una edad de 250 años. El error chico, en cambio, está en todas las filas y no se ve. Una balanza mal calibrada que suma o resta dos kilos al azar produce valores plausibles en todas las mediciones.
- **Sesgo.** Una desviación **sistemática**, que empuja siempre en la misma dirección. Todas las edades son correctas, pero solo contestaron los jubilados. O todos los ingresos declarados son más bajos que los reales, porque nadie declara de más.

La diferencia no es el tamaño ni la cantidad de casos. Es que **el error es aleatorio y el sesgo tiene dirección**.

De ahí sale la consecuencia práctica. El error grosero se puede encontrar en el archivo, porque rompe algún patrón: un hogar de -1 personas se ve, y el bloque 3 del notebook tiene uno. El sesgo casi nunca se ve, porque es consistente con todo lo demás.

Se mide con los diez ingresos declarados de la tabla, que promedian 205.150. Si el instrumento suma o resta hasta 2.000 pesos al azar en cada fila, el promedio se mueve menos de medio punto: los errores se cancelan entre ellos. Si el encuestador visitó solo Córdoba, los cuatro ingresos son correctos y el promedio sube a 230.875, más de un 12%. Esa diferencia no se cancela nunca, porque siempre empuja para el mismo lado.

Y hay un tercer caso, que es el que más veces vas a encontrar y que no entra en ninguna de las dos casillas: el **código de captura**. El 999999 del hogar 9 y el NULL de cobertura_salud no son fallas del mundo, son convenciones del instrumento. Se ven en el archivo, como el error grosero,

y empujan siempre para el mismo lado, como el sesgo. La sección 4.3 los trata aparte, y la 7.5 los encuentra.

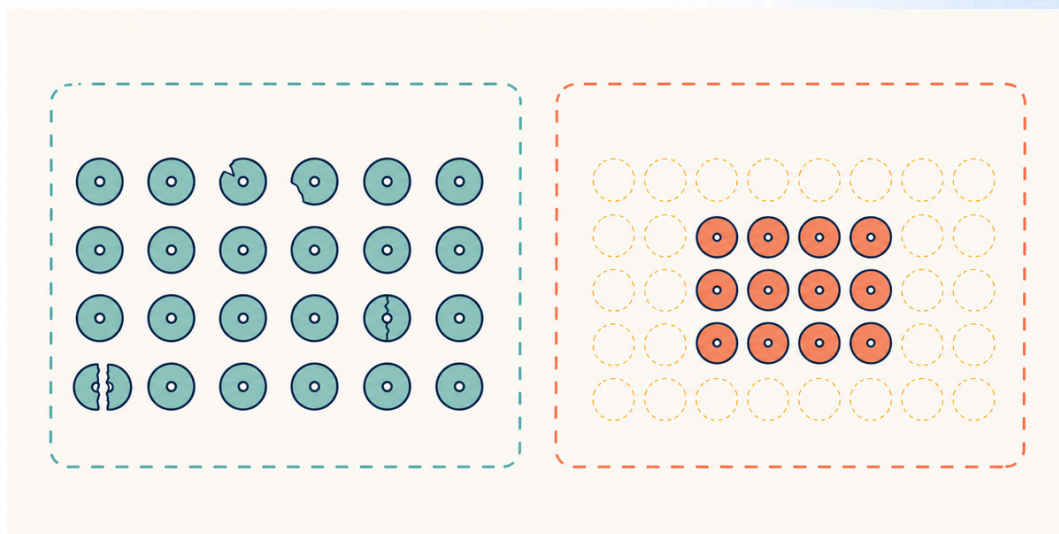


Figura 3. El error está en el valor; el sesgo está en quién quedó adentro y quién quedó afuera.

Por eso este tema va antes del perfilado y no después. El perfilado encuentra errores. El sesgo se detecta sabiendo cómo se capturó el dato, o comparando contra otra fuente.

3.3 Los siete mecanismos

Cinco de estos siete crean el dato. Los otros dos, la API y el scraping, no capturaron nada: transportan lo que capturó otro y le agregan defectos propios encima. La sección 5.2 vuelve sobre eso.

Para cada uno, el sesgo que no se ve y el error que sí:

- **Formulario y encuesta.** *Sesgo:* contesta quien quiere, y dice lo que queda bien. *Error:* texto libre con variantes, unidades mezcladas, fechas ambiguas.
- **Digitalización y OCR.** *Sesgo:* solo se digitalizó lo que sobrevivió en papel. *Error:* 0 por 0, 1 por 1, acentos rotos, columnas corridas.
- **Scraping.** *Sesgo:* solo existe lo que el sitio muestra hoy. *Error:* números con símbolos, filas repetidas, etiquetas de HTML coladas en el texto.
- **API.** *Sesgo:* expone lo que su dueño decidió exponer. *Error:* tope de registros por consulta, campos anidados, cambio de esquema sin aviso.
- **Logs de sistemas.** *Sesgo:* solo se registra lo que alguien programó registrar. *Error:* zonas horarias mezcladas, rotación que borra lo viejo.
- **Sensores.** *Sesgo:* hay sensores donde se pudieron instalar, no donde hace falta medir; y el sensor que se descalibra informa siempre para el mismo lado, sin saberlo. *Error:* lecturas repetidas, saltos imposibles, valor centinela.
- **Carga manual en planilla.** *Sesgo:* refleja lo que el área necesita, no tu análisis. *Error:* celdas combinadas, colores como dato, totales en el medio de los datos.

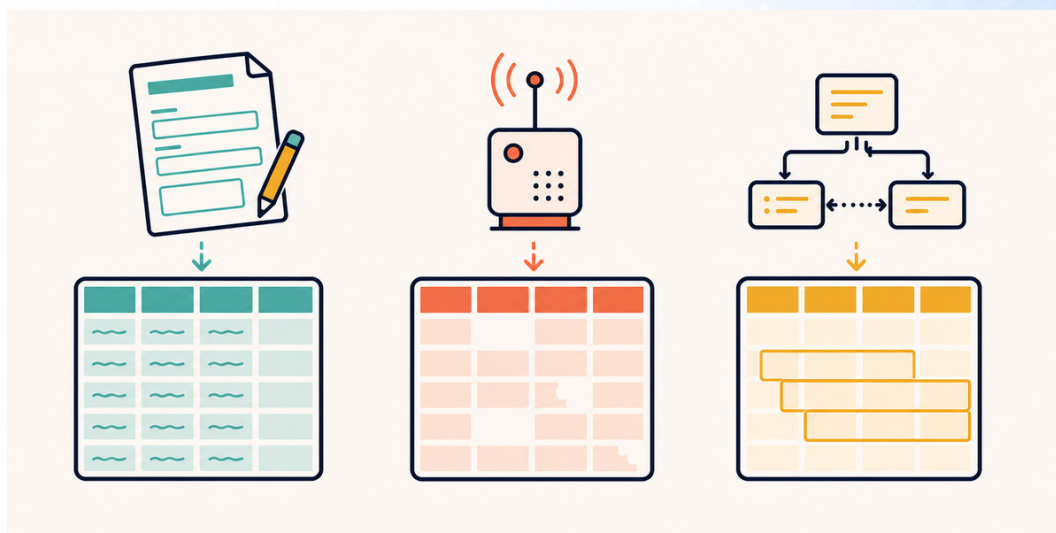


Figura 4. Cada instrumento deja una huella distinta, y esa huella se puede anticipar.

Los errores se detectan todos con la misma rutina, que es la de la sección 7. Los sesgos no se detectan ahí, y eso es el punto. Tres de los que figuran como error son en realidad sesgos disfrazados: una API con tope de 1.000 registros y 12.000 hogares no devuelve mil hogares al azar, devuelve siempre los mil primeros, y la rotación de los logs se lleva siempre los más viejos.

3.4 El formulario, que es el que más vas a ver

De los siete, el formulario es el que más problemas produce, porque es el único donde una persona improvisa la respuesta en el momento.

PREGUNTA MAL DISEÑADA	LO QUE APARECE EN TU TABLA
Campo de texto libre para la localidad	→ "Ramallo", "ramallo", "Ramallo ", "Concep.", "CONCEPCIÓN"
"Ingresos: ____" sin unidad ni formato	→ "185000", "\$185.000", "185.000,50", "no contesta"
"Fecha: __/__/____" sin decir el orden	→ 03/04/2025 · ¿3 de abril o 4 de marzo? No hay forma de saberlo
"Servicios que tiene:" una sola casilla de texto	→ "agua, luz", "agua;luz", "todos", "agua y luz" – varios en una celda
Pregunta que no aplica a todos, sin ruta de salto	→ Un código raro, o vacío, o el valor anterior repetido

Cada flecha de esa lista es un problema que después alguien tiene que arreglar a mano. Arreglarlo en el formulario cuesta cinco minutos. **Arreglarlo después cuesta días**, y algunos casos no se pueden arreglar nunca: de 03/04/2025 no se recupera la fecha verdadera si no hay otro dato que la desambigüe.

El primer caso de ese cuadro explica la columna `localidad` de nuestra tabla, y el cuarto explica la columna `servicios`. El quinto va a explicar, en la sección 7, los ocho `NULL`. El segundo y el tercero no están en la tabla, y se ven corriendo el bloque 3 del notebook.

3.5 La regla de escribir cómo se capturó

Todo lo anterior se resume en una sola instrucción:

Antes de abrir un conjunto de datos, escribí en una línea cómo se capturó. Después buscá en la tabla los errores típicos de ese mecanismo. Vas a encontrar la mayoría.

Aplicada al relevamiento de esta clase: **encuesta a hogares, cargada por encuestadores en terreno con un formulario digital.**

Con ese mecanismo esperamos tres cosas: texto libre escrito de varias formas, varios valores en una celda y preguntas que no se contestaron. Las tres están, y la sección 7 las encuentra en orden.

4. DISEÑAR LA CAPTURA

Todo lo anterior mira el dato hacia atrás. Esta sección lo mira hacia adelante: qué se hace para que el problema no exista.

Es la parte del taller que más rinde en el trabajo. Casi nadie limpia datos por gusto: se limpia porque nadie escribió las cinco columnas de la sección que sigue antes de preguntar.

4.1 La tabla de captura

El diseño de la captura se escribe antes de preguntar, y se escribe como una tabla. No es la lista de preguntas del formulario: es la definición de cada variable que el formulario va a producir. Para cada una, cinco columnas.

variable	tipo	dominio	obligatoria	validación
personas	entero	1 a 20	sí	si vale 1, no puede haber menores
satisfaccion	opciones	mala, regular, buena	no	no preguntar si no usa el centro

La última columna dice qué se comprueba mirando **dos** variables a la vez. Sin esa regla, el formulario acepta un hogar de una persona que declara tres menores: los dos valores están en su dominio, cada campo pasa su validación por separado, y la combinación no existe en el mundo.

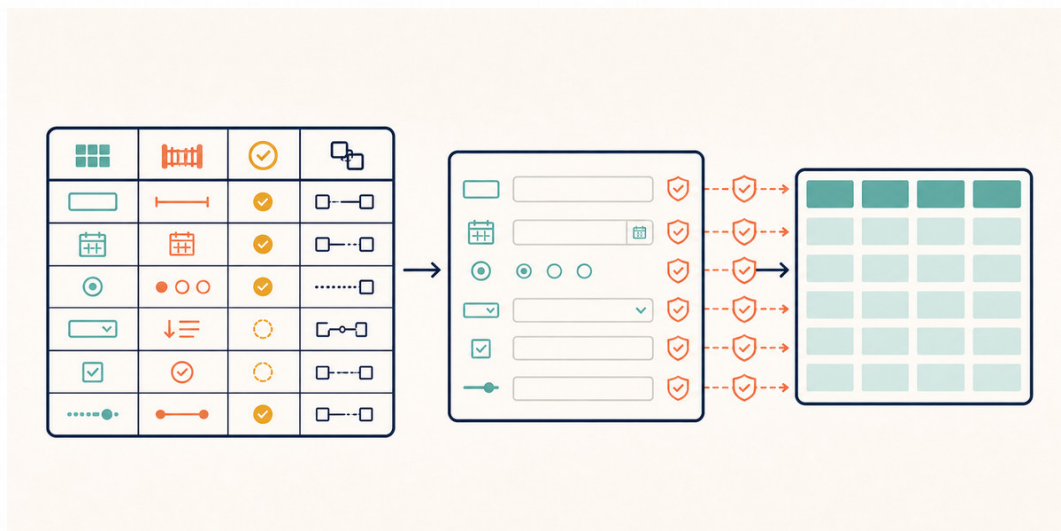


Figura 5. La calidad se diseña antes de preguntar: lo que el formulario no valida se paga después.

4.2 Las cinco reglas del diseño

- **Nada de texto libre**, salvo que se pueda justificar por qué es imprescindible. La columna localidad de nuestra tabla tendría que haber sido una lista de tres opciones. Con una lista, los cinco valores distintos serían tres.
- **Cada variable con su dominio explícito**: la lista de valores admitidos, o el mínimo y el máximo. Sin dominio, el 999999 de ingreso entra sin que nada lo frene.
- **Definir qué pasa cuando la pregunta no aplica**. Es la sección 4.3, y es la regla que más cuesta.
- **Al menos una validación entre campos**, o sea una regla que mira dos variables a la vez, como las dos del ejemplo. Un formulario que valida cada campo por separado deja pasar combinaciones imposibles.
- **Si una pregunta admite varias respuestas**, decidir cómo se guarda **antes**, sabiendo que un valor por celda es la regla. La columna servicios es lo que pasa cuando esa decisión no se toma.

La columna dominio es la misma información que en la sección 6.5 se va a llamar valores_admitidos. La tabla que diseña la captura y el diccionario de variables son el mismo documento en dos momentos de la vida del dato.

4.3 Las tres ausencias: no aplica, no contestó, no se preguntó

Una celda vacía puede significar tres cosas distintas, y las tres se ven exactamente iguales:

- **No aplica**. La pregunta no correspondía a este hogar. No falta nada: la respuesta correcta es que no hay respuesta.
- **No contestó**. La pregunta correspondía y la persona no quiso contestar. Eso es un dato, y además es informativo: el que no declara su ingreso rara vez es un hogar cualquiera.
- **No se preguntó**. La pregunta correspondía y el instrumento falló. Se saltó una hoja, se cortó la tablet, el encuestador se olvidó.

Un formulario bien diseñado usa tres códigos distintos para esos tres casos. Uno solo los vuelve indistinguibles para siempre.

Tres códigos quiere decir literalmente esto:

-9	no aplica	la pregunta no correspondía a este hogar
-8	no contestó	correspondía, y la persona no quiso contestar
-7	no se preguntó	correspondía, y falló el instrumento

Al leer el archivo, los tres se declaran como faltantes en una línea, como en el notebook de la clase 1:

```
pd.read_csv(ruta) # los tres entran como -9, -8 y -7, y se promedian
pd.read_csv(ruta, na_values=[-9, -8, -7]) # los tres entran como faltantes, y ya son iguales
```

La segunda línea es la correcta, y es también la que borra la distinción. Por eso el diccionario se escribe antes: después de esa línea, los tres casos son el mismo. Un cuestionario de encuesta real separa cuatro, porque "no sabe" y "no contesta" tampoco son lo mismo.

Esto no es una sutileza. La decisión de qué hacer con un faltante depende enteramente de cuál de los tres es, y esa información solo la tiene quien diseñó la captura. Si no la escribió, se perdió el día que se cerró el relevamiento, y en la sección 7.5 vamos a chocar con ese límite.

5. TRAER EL DATO DE OTRAS FUENTES

La clase 1 leyó un CSV y una planilla de Excel. Un dato también puede llegar de otras dos formas, y las dos se leen con la misma librería.

5.1 Una librería para todas las fuentes

Fuente	Función
CSV	<code>pd.read_csv</code>
Planilla	<code>pd.read_excel</code>
Respuesta de una API	<code>pd.read_json</code>
Tabla en una web	<code>pd.read_html</code>

La dificultad casi nunca está en la función. Está en lo que viene adentro.

Las cuatro aceptan una URL igual que aceptan una ruta de archivo. Los argumentos, en cambio, no son los mismos. `read_csv` y `read_excel` comparten casi todos. `read_html` no acepta `dtype`. Y `read_json` no acepta ninguno de los que sirven para los faltantes y para los formatos numéricos: no tiene `na_values`, ni `decimal`, ni `thousands`. Con un JSON esa limpieza se hace después. La lista de cada una está en [IO tools](#), la documentación de pandas.

Por eso vale la misma regla de la clase 1, y ahora con más razón:

Después de leer, siempre se mira. `shape` primero, `head()` después, `dtypes` siempre. Una lectura que no explota no es una lectura correcta.

5.2 Qué es una API, y por qué no es un mecanismo de captura

Una **API web** (de *Application Programming Interface*) es una dirección pensada para que la consuma un programa y no una persona: devuelve datos en lugar de una página para mirar.

El mecanismo es el mismo que usa el navegador. Tu programa pide una dirección, el servidor contesta con un código y con el contenido, casi siempre en formato **JSON**, que es el formato semiestructurado de la clase 1, sección 3.2.

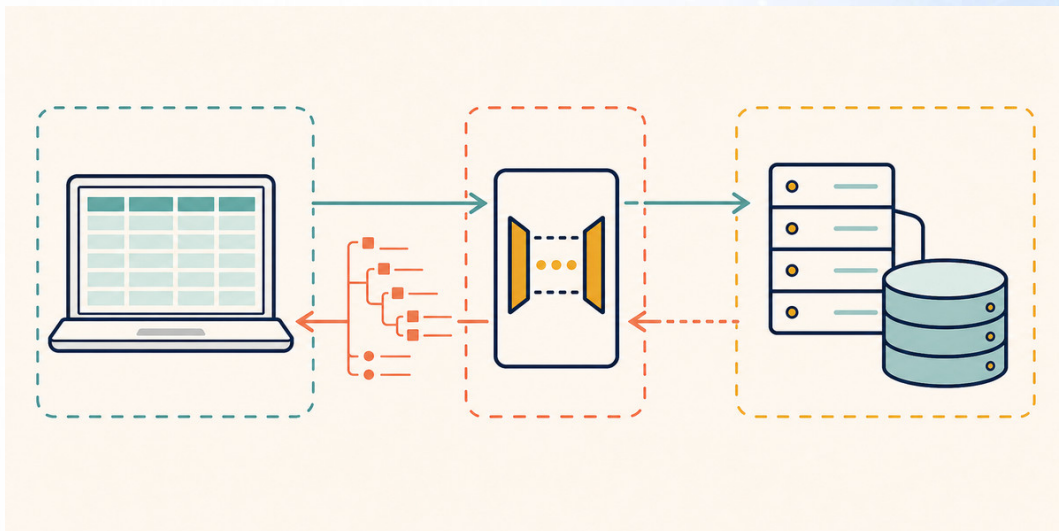


Figura 6. La API expone y transporta el dato; no necesariamente fue el mecanismo que lo creó.

Dos advertencias, y la segunda es la que importa para esta clase.

La primera es que un archivo que descargaste queda fijo, y **una dirección que consultás se vuelve a evaluar cada vez**. Eso vale igual para un CSV publicado en una URL que para una API: la misma dirección puede devolver algo distinto mañana. Es una ventaja y es un riesgo, y va anotado en la documentación del análisis.

La segunda: **una API casi nunca es el mecanismo que capturó el dato**. Es la vía por la que el dato te llega. Detrás de esa dirección hay un formulario, un sensor o una planilla, y los errores que vas a encontrar son los de ese instrumento, no los de la vía. Por eso dos respuestas pueden convivir: el dato se capturó con un formulario y vos lo recuperaste mediante una API.

5.3 Una tabla web es una presentación, no un conjunto de datos

`pd.read_html` busca **todas** las etiquetas `<table>` de una página y devuelve una **lista** de tablas, aunque haya una sola. Siempre hay que elegir con un índice.

Es la forma más frágil de las cuatro, por cuatro motivos:

1. La página puede cambiar mañana, y tu índice ya no apunta a la misma tabla.
2. Una tabla hecha para que la lea una persona usa recursos que rompen una tabla de datos: encabezados en dos pisos, celdas combinadas, notas al pie adentro del título, filas de totales mezcladas con los datos. Es la planilla desprolija de la clase 1, sección 3.6, otra vez.
3. Los números vienen formateados para leer y no para calcular. `read_html` supone que la coma separa los miles, así que un 41,67 vuelve como el entero 4167, cien veces más grande, en una columna que `dtypes` no delata porque es numérica. Se arregla al leer, con `pd.read_html(pagina, decimal=",", thousands=None)`. Si la celda además trae el signo %, la columna vuelve como texto, y ahí el problema se ve.
4. Solo ve el HTML que manda el servidor. Una tabla que dibuja JavaScript después de cargar la página no está, y ahí `read_html` avisa que no encontró ninguna tabla.

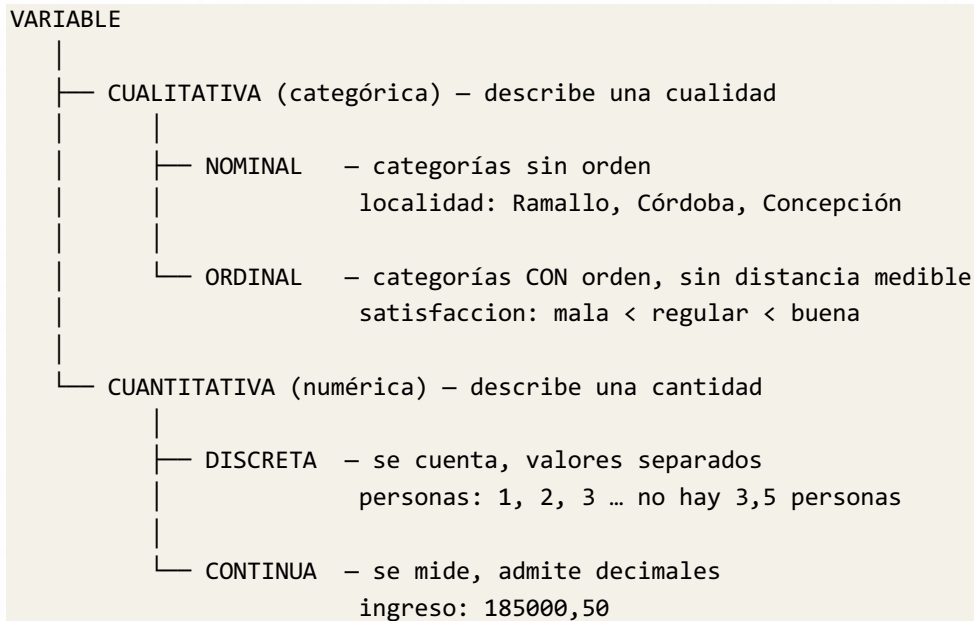
6. QUÉ ES CADA VARIABLE

La pregunta de esta sección es práctica, no académica:

¿Qué puedo calcular y qué puedo graficar con esta columna?

La respuesta no depende del tipo que eligió pandas. Depende de **qué representa** la variable en el mundo. La columna `id` de nuestra tabla es una columna de enteros. Su promedio es 7,25, pandas lo calcula sin protestar, y no significa nada.

6.1 Cualitativa y cuantitativa



Las dos ramas cualitativas se corresponden con las dos primeras escalas de la sección que sigue, y la rama cuantitativa con las dos últimas. Discreta y continua es otra pregunta, que se hace sobre esas dos últimas: decide el gráfico, y eso es tema de la clase 3. Una fecha entra por la rama cuantitativa y es discreta, porque se cuenta en días.

6.2 Las cuatro escalas de medición

Las escalas son acumulativas: cada una permite todo lo de la anterior y algo más.

Qué comparás	Nominal	Ordinal	Intervalo	Razón
¿Hay orden?	no	sí	sí	sí
¿La distancia significa algo?	no	no	sí	sí
¿El cero es "nada"?	no aplica	no aplica	no	sí
¿Se puede promediar?	no	no	sí	sí
¿Se puede dividir?	no	no	solo las diferencias	los valores
Estadístico central	moda	mediana	media	media
En nuestra tabla	localidad	satisfaccion	fecha_visita	personas

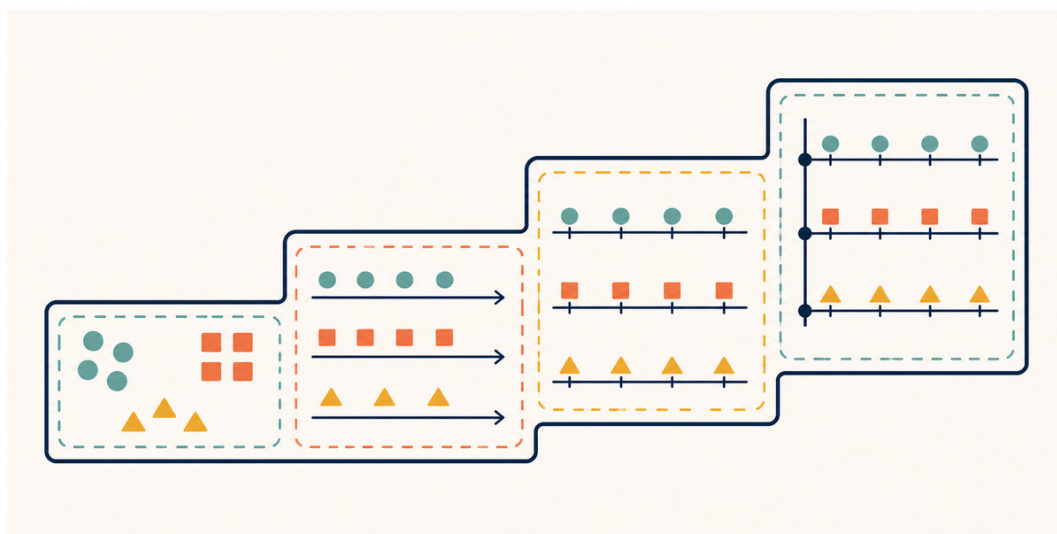


Figura 7. Subir un escalón nunca quita lo que permitía el anterior.

6.3 Las dos confusiones que hay que evitar

Intervalo contra razón: el cero. En una escala de intervalo el cero es un punto elegido por convención. `fecha_visita` está en una escala de intervalo. Restar dos fechas da algo con sentido: del 2025-03-14 al 2025-03-21 pasaron siete días. Dividir dos fechas no significa nada, porque el origen del calendario es una convención y no un punto donde no hay tiempo. Dividir dos **duraciones** sí: catorce días son el doble de siete, y eso ya es una escala de razón.

Con `ingreso` es distinto desde el principio: cero pesos es no tener plata, así que un ingreso de 240.000 sí es el doble de uno de 120.000. Esa es una escala de razón.

Promediar, en cambio, sí se puede en las dos. El promedio de las doce fechas de visita es el 17 de marzo a las ocho de la mañana: un momento en el que nadie visitó nada, y un resumen correcto igual.

Ordinal: la trampa de codificar y promediar. Si codificás mala=1, regular=2, buena=3, las ocho respuestas de nuestra tabla son 1 1 2 2 2 3 3 3, y su promedio es 2,1. Ese número **no significa nada**, y encima no está en la lista: estás suponiendo que la distancia entre "mala" y "regular" es igual a la distancia entre "regular" y "buena". Nadie midió esas distancias: son etiquetas de un cuestionario.

Para una ordinal se usan moda y mediana. La moda de satisfaccion es un empate entre "buena" y "regular", con tres cada una. La mediana rompe ese empate y da "regular", que es el valor que parte las ocho respuestas al medio. Las dos cosas sí se pueden afirmar. Eso sí: pandas no calcula la mediana de una columna de texto, porque no sabe el orden; hay que declararlo antes, y eso es tema de la clase 3.

Esto pasa todo el tiempo con las escalas de satisfacción de 1 a 5. Hay quien lo defiende, y la discusión sigue abierta. Lo que no se discute es que la decisión se toma a sabiendas y se deja escrita.

6.4 El tipo de pandas no es la escala

*El tipo te dice **cómo lo guardó la computadora**. La escala te dice **qué tenés derecho a calcular**.*

No son lo mismo, y la escala **no está en el archivo**: la ponés vos. Tres columnas de nuestra tabla lo muestran:

- id es un entero para pandas y no es una variable de análisis: es una etiqueta.
- satisfaccion es texto para pandas, igual que localidad, y las dos escalas son distintas.
- fecha_visita es texto para pandas y es una escala de intervalo.

Y como la escala no está en el archivo, hay que escribirla en algún lado. Ese lado es el diccionario.

6.5 El diccionario de variables

Un **diccionario de variables** es la tabla que dice, para cada columna, qué significa y qué valores puede tomar. Los **metadatos** son la información sobre el conjunto completo: quién lo produjo, cuándo, con qué método y bajo qué licencia. Es la ficha de la clase 1, sección 4.1, y el estándar que sigue datos.gob.ar es el [Perfil Nacional de Metadatos](#).

Sin diccionario, un conjunto de datos es una tabla de números que nadie puede auditar. Es el diccionario de datos de la clase 1, sección 4.1, con un nombre más preciso, una fila por variable, y es la **trazabilidad** de su sección 3.7 hecha documento.

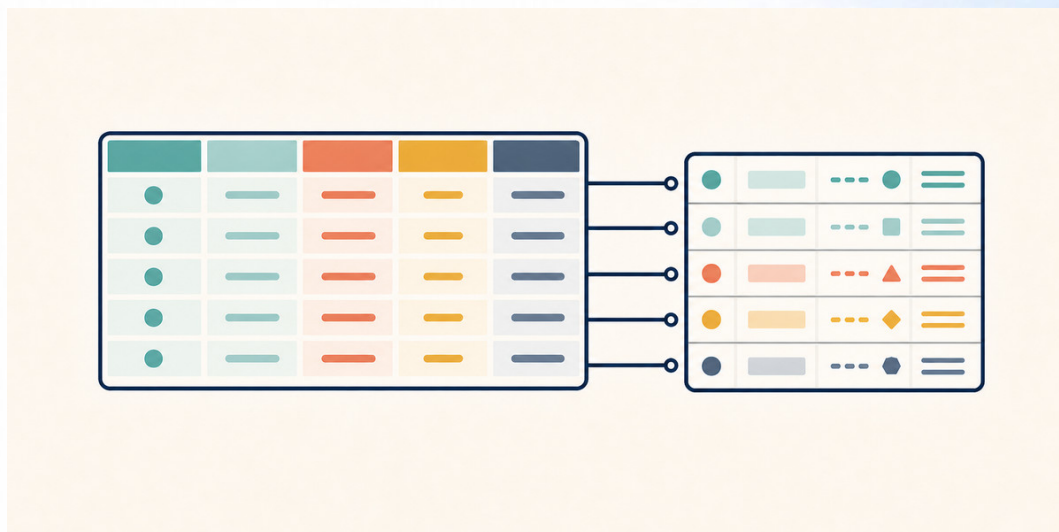


Figura 8. El conjunto de datos guarda los valores; el diccionario guarda lo que significan.

Las ocho columnas mínimas del diccionario, con lo que contesta cada una:

Campo	Qué responde
variable	El nombre exacto
descripcion	Qué mide
tipo	Cualitativa o cuantitativa
escala	Cuál de las cuatro
unidad	En qué está expresada
valores_admitidos	El dominio
codigo_faltante	Cómo se marca
origen	De qué pregunta sale

Una entrada completa, la de la columna más difícil de las ocho:

- **variable:** satisfaccion.
- **descripcion:** conformidad declarada con la atención del centro de salud que usa el hogar. No repite el nombre: dice qué mide y de quién.
- **tipo:** cualitativa.
- **escala:** ordinal. Hay orden y no hay distancia.
- **unidad:** no tiene. Es una categoría.
- **valores_admitidos:** mala, regular, buena, en ese orden.
- **codigo_faltante:** el texto NULL, y no sabemos qué significa. Eso también se escribe.
- **origen:** pregunta 7 del cuestionario, que no tenemos.

Fijate que la escala y las cuatro últimas son las que hacen el trabajo, y pandas no puede sacar ninguna.

6.6 Por qué se escribe antes y no después

Tres razones, de peso creciente.

Antes es diseño; después es adivinanza. Si el diccionario se escribe antes de capturar, la columna `valores_admitidos` se convierte en la validación del formulario. La misma tabla que documenta, define. Si se escribe después, hay que **adivinar** el dominio a partir de los valores que llegaron.

Sin dominio previo no podés distinguir un error de un valor raro. Un valor solo es imposible contra una definición. Nuestro ingreso de 999999 es sospechoso porque sabemos qué ingresos son plausibles en esos hogares. Si el dominio se escribe mirando los datos, todo valor presente resulta posible, y el diccionario no detecta nada.

Es la única defensa contra las tres ausencias de la sección 4.3. Solo el diccionario, escrito cuando se diseñó la captura, puede decir cuál de las tres es cada celda vacía.

Y esto no es solo una exigencia del taller. El [Perfil Nacional de Metadatos](#) define una clase `Field` para documentar cada columna de un archivo tabular. Es una clase recomendada y no obligatoria, y ese es exactamente el problema: el diccionario es lo primero que se cae cuando no se exige.

7. ANÁLISIS DE LA INFORMACIÓN DE ENTRADA

El **perfilado** es el diagnóstico que se hace antes de tocar nada. Todavía no cambiamos nada: solo miramos qué tiene la tabla.

Existen librerías que generan un informe de perfilado completo en una línea. En este taller no las usamos, y la razón es concreta:

*Una herramienta de perfilado te dice **dónde mirar**. No te dice **qué está mal**.*

El informe automático te muestra que `localidad` tiene cinco valores distintos. No te puede decir que tres de ellos son la misma localidad, ni cuál de los cinco es el nombre correcto. Esa distinción necesita saber cómo se capturó el dato, que es la sección 3.

7.1 De cinco preguntas a seis pasos

La rutina del perfilado sale de las cinco preguntas de la clase 1, con dos cambios. Los nombres de las columnas dejan de ser un paso aparte y entran en el paso de tipos. Y se agregan dos pasos: el resumen y la cardinalidad.

1. TAMAÑO	<code>shape</code>	¿cuántas filas y columnas?
2. TIPOS	<code>dtypes</code>	¿pandas leyó bien cada columna?
3. RESUMEN	<code>describe()</code>	¿los rangos son posibles?
4. FALTANTES	<code>isna().sum()</code>	¿cuánto falta y dónde?
5. DUPLICADOS	<code>duplicated().sum()</code>	¿hay filas repetidas?
6. CARDINALIDAD	<code>nunique, value_counts</code>	¿cuántas categorías y cuáles?

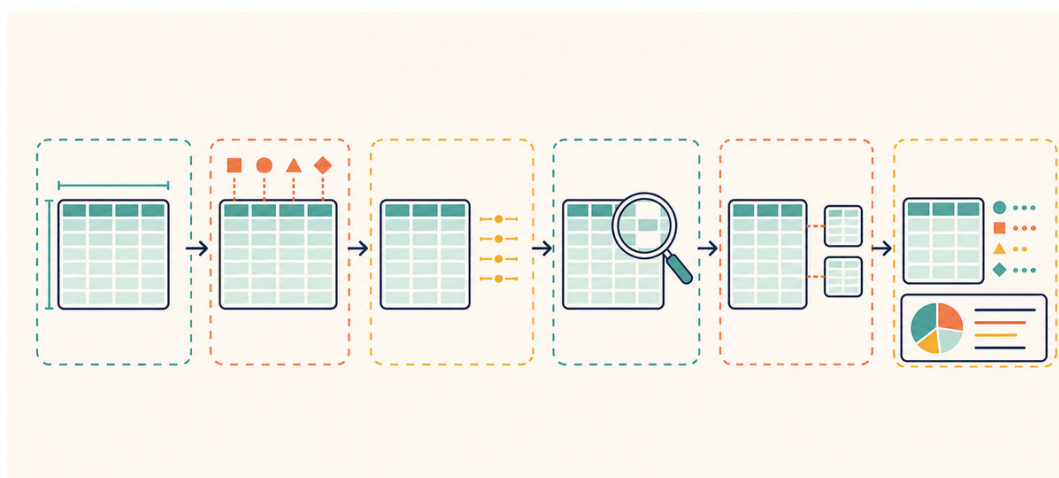


Figura 9. Perfilar es inspeccionar en un orden estable antes de tomar decisiones.

Sobre nuestra tabla, los seis pasos contestan esto:

1. Tamaño. Doce filas y ocho columnas, que es lo que esperábamos.

2. Tipos. ingreso volvió como decimal y no como entero, porque el tipo entero de pandas no tiene forma de escribir un faltante: es lo mismo que pasaba en la clase 1. fecha_visita volvió como texto, que pandas imprime str o object según la versión. Y id volvió como entero, aunque sea una etiqueta.

3. Resumen. Acá aparecen los dos primeros hallazgos.

El máximo de ingreso es 999999. No es un ingreso: es un **valor centinela**, o sea un número que alguien eligió para decir "no sé", como el 999 de la clase 1, sección 4.3. Y cuesta caro, porque la columna sigue siendo numérica y ese valor se promedia. Con el centinela adentro, el ingreso promedio es 277.409. Sin él, 205.150. El promedio sucio es un 35% más alto que el limpio, sin ningún error a la vista.

El máximo de id es 14 y hay doce filas. La numeración **tiene huecos**: dos encuestas se perdieron o se descartaron. Eso es información sobre el relevamiento, no sobre los hogares.

4. Faltantes. Uno en ingreso, cuatro en cobertura_salud, cuatro en satisfaccion, cero en las otras cinco columnas.

5. Duplicados. Ninguna fila repetida y ningún id repetido. Las dos preguntas son distintas y las dos hay que hacerlas: que no haya filas idénticas no garantiza que no haya un hogar cargado dos veces con una diferencia mínima.

6. Cardinalidad. El paso nuevo, y el de la sección que sigue.

7.2 Cardinalidad: cuándo una columna es una categoría

La **cardinalidad** de una columna es la cantidad de valores distintos que tiene. Sirve para decidir qué es cada columna, comparando ese número contra la cantidad de filas.

- **Pocos valores distintos y muchas filas:** es una categoría. satisfaccion tiene tres, cobertura_salud tiene dos, aunque en el archivo haya un tercer texto escrito que la sección 7.5 encuentra. Una variable de dos categorías se llama **dicotómica**.
- **Tantos valores distintos como filas:** es un identificador. id tiene doce en doce filas. Contar sus categorías no informa nada.
- **Un número intermedio y raro:** casi siempre es un problema. Y acá hay dos.

localidad tiene cinco valores distintos y hay tres localidades. Ramallo, ramallo y Ramallo con un espacio al final son el mismo pueblo escrito de tres formas. El conteo de valores las muestra como tres filas separadas. La segunda se distingue a la vista, por la minúscula. La primera y la tercera se ven idénticas en la pantalla: la diferencia es un espacio que no se dibuja. Es el precio del campo de texto libre de la sección 3.4.

servicios tiene seis valores distintos en doce filas. Pero no hay seis servicios: hay cuatro, que son agua, luz, gas y cloacas. Los seis valores son combinaciones de esos cuatro. Esa es la firma de una columna **multivalor**, y rompe la regla de un valor por celda.

Los dos se **diagnostican hoy y no se tocan**: se arreglan en la clase 3.

7.3 Los faltantes tienen patrón

El paso 4 dijo cuatro y cuatro. Y acá está la trampa de esta clase:

"La misma cantidad" no es lo mismo que "las mismas filas".

Podrían ser cuatro hogares distintos en cada columna y el conteo daría igual. Que las dos columnas tengan cuatro faltantes es un dato. Que sean **las mismas cuatro filas** es otro, y es el que explica algo.

Con doce filas se ve a ojo: son los hogares 3, 5, 10 y 13, y en las dos columnas. Con doce mil filas no se ve, y hay que contarlo. La forma de contarlo es la de la sección siguiente, y es la herramienta que más se usa en todo el taller.

7.4 La herramienta nueva: quedarse con un subconjunto de filas

Para preguntar "de las filas donde falta `cobertura_salud`, cuántas tampoco tienen `satisfaccion`", hace falta quedarse con un grupo de filas. Eso se llama **indexado booleano**, y es la única herramienta nueva de esta clase. *Booleano* quiere decir de dos valores, verdadero o falso: viene del matemático George Boole, y en la salida de pandas aparece como `bool`.

Paso 1. Una comparación no devuelve sí o no: devuelve una columna.

`df["personas"] > 4` no contesta una sola cosa. Compara **fila por fila** y devuelve una columna nueva, del mismo largo que la tabla, con `True` donde la comparación se cumple y `False` donde no. Entran doce filas, salen doce valores de verdadero o falso.

Esa columna no es el resultado que buscás: es el **filtro** con el que lo vas a pedir. `df["cobertura_salud"].isna()` hace exactamente lo mismo y devuelve lo mismo. Solo cambia la pregunta.

Paso 2. Por qué esa columna se puede meter adentro de los corchetes.

Los corchetes de `df[...]` hacen dos cosas distintas, según lo que les pongas adentro:

Lo que va adentro	Qué devuelve
Una lista de nombres	Esas columnas
Una columna de <code>True</code> y <code>False</code>	Esas filas

El mismo símbolo para dos operaciones distintas. Es la confusión más común al empezar, y la regla corta es: **los nombres eligen columnas, True y False eligen filas.**

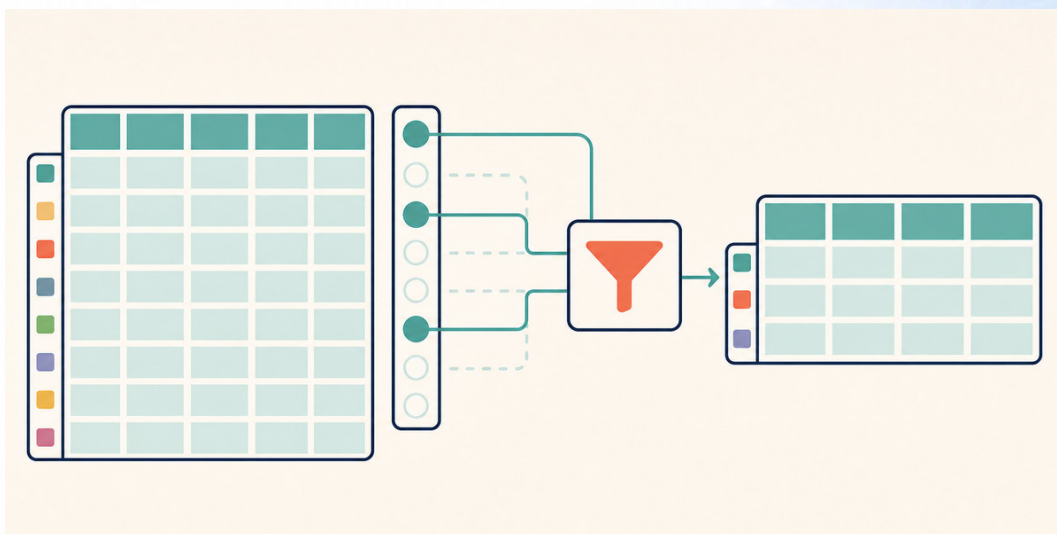


Figura 10. El filtro decide qué filas pasan; las columnas quedan todas.

Paso 3. El paso intermedio, escrito.

La forma de una sola línea mete la comparación adentro de los corchetes. Escribirla en dos líneas da el mismo resultado y deja ver el filtro:

```
comparacion = df["personas"] > 4 # el filtro: 12 valores de True/False
df[comparacion]                  # las filas donde el filtro es True
```

Sobre nuestra tabla eso devuelve tres hogares, los de id 2, 7 y 11. Dos cosas de ese resultado:

- **La cantidad de True es la cantidad de filas que sobreviven.** Por eso sumar el filtro sirve para contar sin filtrar.
- **El filtro saca filas, nunca columnas.** Las ocho columnas quedan intactas.

Aplicado a la pregunta de la sección anterior: nos quedamos con las filas donde falta cobertura_salud, que son cuatro, y contamos cuántas de **esas** tampoco tienen satisfaccion.

Cuatro de cuatro. No es casualidad: es una sola causa. Las dos preguntas se dejaron sin contestar siempre juntas, nunca una sin la otra. Un faltante al azar no se comporta así.

7.5 De la evidencia a la hipótesis de captura

Sabemos **qué** pasó. Ahora hay que averiguar **por qué**, y para eso se descartan las dos sospechas naturales: fue en una localidad, o fue en una época.

No se ve concentración por localidad. Los cuatro hogares están en las tres localidades: uno en Concepción, dos en Ramallo y uno en Córdoba. Ninguna concentra el problema. Ojo con el conteo en el notebook: los dos de Ramallo salen en filas separadas, porque uno está escrito ramallo. Es el mismo defecto de la sección 7.2, que todavía no arreglamos.

Ni por fecha. Las visitas de esos cuatro van del 15 al 20 de marzo, y el relevamiento entero va del 14 al 21. Los faltantes cubren casi todo el período, así que no hubo un tramo afectado. Con cuatro casos esto no descarta ninguna de las dos causas: solo dice que ninguna salta a la vista.

Queda una fuente de evidencia sin usar: **el archivo, leído sin que pandas lo interprete**. Se lee con `pd.read_csv(..., keep_default_na=False)`, que le dice a pandas que no tome ningún texto como ausencia. Es la regla del dato crudo intacto de la clase 1, sección 3.8, y para esto existe.

Leído así, las dos columnas no están vacías: dicen el texto NULL, cuatro veces cada una, **en las mismas cuatro filas**. `read_csv` reconoce NULL como ausencia y lo convierte solo. Por eso el conteo del paso 4 lo mostró como un faltante, y perdimos la única pista que había.

Y la comparación cierra el argumento. En el hogar 3, ingreso está **realmente vacío**: nadie escribió nada. En esa misma fila, `cobertura_salud` dice NULL. Dos ausencias distintas en la misma tabla, y una de las dos la escribió a propósito algún programa de la cadena: el que capturó, o el que exportó el archivo. Con lo que tenemos no se puede saber cuál.

Juntemos las cuatro cosas que sabemos:

- **Faltan las dos, en las mismas cuatro filas.** Lo dio el conteo de cuatro de cuatro.
- **No se concentran en una localidad.** Lo dio el reparto por localidad.
- **No se concentran en un tramo del período.** Lo dio el rango de fechas.
- **No son celdas vacías: son el código NULL.** Lo dio el archivo crudo.

Un dato que se pierde por accidente no se comporta así. Se perdería a veces en una columna y a veces en la otra, y no dejaría un código escrito. Quedan cuatro explicaciones en pie, y las cuatro son sobre el instrumento y no sobre los datos:

1. **Salto condicional en el formulario.** Una pregunta anterior, del tipo "¿usa el centro de salud municipal?", decidía si se mostraban estas dos. A cuatro hogares no se les mostró, y el sistema grabó NULL en las dos. Esa pregunta previa no está en el archivo exportado.
2. **Módulo opcional.** Las dos preguntas eran una sección que el encuestador podía omitir.
3. **Un solo encuestador, o una sola tablet.** No se puede comprobar, porque la tabla no tiene ninguna columna que identifique al encuestador. Esa ausencia es un hallazgo en sí misma, y es exactamente lo que la sección 4 pedía diseñar.
4. **Artefacto de la exportación.** El texto NULL es la firma típica de un volcado de base de datos: si las dos preguntas viven en otra tabla, los hogares sin fila coincidente salen con NULL al exportar, y el formulario nunca escribió nada.

Una quinta explicación es la única que la evidencia descarta: que la sección se haya **agregado a mitad del relevamiento**. Si fuera así, los cuatro casos estarían al principio del período, y están repartidos en todo el rango.

Entre las cuatro que quedan no podemos elegir con los datos que tenemos. Para eso hace falta el cuestionario, o el diccionario de variables. Ninguno de los dos existe.

Y este es el puente con la sección 4.3. Si quien diseñó este relevamiento hubiera usado tres códigos en vez de un NULL para todo, la pregunta estaría contestada en un segundo. Con un solo código, no tiene respuesta.

La conclusión de la clase, en una línea. Los faltantes no son ruido que hay que sacar. Son un dato sobre el instrumento de captura, y a veces son el único que queda.

7.6 Lista de verificación

Cuando recibas un conjunto de datos que no conocés, en este orden:

1. **Escribí cómo se capturó**, en una línea. Si no lo sabés, preguntalo. Eso te dice qué errores buscar.
2. **Pedí el diccionario y la ficha**. Sin ellos, todo lo que sigue son supuestos tuyos.
3. **Corré los seis pasos**, siempre en el mismo orden.
4. **Para cada columna, escribí la escala**. No el tipo: la escala.
5. **Mirá la cardinalidad de las columnas de texto**. Un número intermedio y raro es un problema.
6. **Con los faltantes, contá filas y no solo cantidades**. Dos columnas con el mismo número son una pista, no una conclusión.
7. **Volvé al archivo crudo** antes de decidir qué es un faltante.
8. **Anotá todo lo que supusiste**. Cada supuesto es tuyo y no un hecho del archivo.

Los ocho puntos son diagnóstico. Ninguno modifica el dato: eso es la clase 3.

8. LOS EJEMPLOS EN GOOGLE COLAB

Casi todo lo que describe este apunte está también en un notebook, con el código ya escrito y listo para ejecutar: [02 captura y variables.ipynb](#). La tabla de doce filas se fabrica ahí mismo, así que no depende de ningún portal. Una sola celda pide internet, y está avisada: si no contesta, se sigue con la siguiente.

Una celda del notebook falla a propósito: es la que intenta promediar *satisfaccion*, que es ordinal. Está avisada antes de llegar. Cuando aparezca el bloque rojo, seguí con la celda siguiente.

Los seis bloques que vas a ejecutar, y a qué sección de este apunte corresponde cada uno:

1. **El relevamiento, fabricado y mirado.** La tabla de doce filas, primero como texto crudo y después leída por *pandas*. Secciones 1.3 y 3.5.
2. **Las cuatro formas de traer un dato.** El CSV, la planilla, el JSON y la tabla web, cada una en una celda. Secciones 5.1 a 5.3.
3. **El formulario sin diseño.** Cinco vecinos que contestan lo mismo de cinco formas distintas, y qué llega a la tabla. Secciones 3.4 y 4.2.
4. **El diccionario, a medias.** La mitad que saca *pandas* sola y la mitad que exige criterio. Sección 6.5.
5. **Los seis pasos del perfilado.** La rutina entera sobre el relevamiento, en el orden fijo. Sección 7.1.
6. **El caso de los ocho NULL.** El indexado booleano, el conteo de cuatro de cuatro, la vuelta al archivo crudo y el sesgo medido con el filtro. Secciones 7.3 a 7.5, y 3.2.

Quedan afuera del notebook, porque se entienden leyendo, los siete mecanismos de captura y las cinco reglas del diseño de la captura. Todo eso está en este apunte.